Model Name	Description	Advantages	Disadvantages	Best Use Cases
Linear Regression	Basic model assuming linear relationship between features and target. Uses OLS (Ordinary Least Squares).	Simple and interpretable; Fast training and prediction; Works well with linearly separable data; Low computational cost	Assumes linearity; Sensitive to outliers; Cannot capture complex patterns; Prone to multicollinearity	Simple prediction tasks; Feature importance analysis; Baseline model; Linear relationships
Ridge Regression (L2)	Linear regression with L2 regularization penalty to prevent overfitting.	Reduces overfitting; Handles multicollinearity well; Keeps all features; Stable predictions	Still assumes linearity; Doesn't perform feature selection; Requires hyperparameter tuning	High multicollinearity; Many correlated features; Preventing overfitting
Lasso Regression (L1)	Linear regression with L1 regularization that can shrink coefficients to zero.	Performs feature selection; Reduces overfitting; Creates sparse models; Interpretable results	May select only one from correlated features; Unstable with highly correlated features; Still linear	Feature selection needed; High-dimensional data; Sparse solutions desired
ElasticNet	Combines L1 (Lasso) and L2 (Ridge) regularization.	Best of both worlds; Handles correlated features better than Lasso; Feature selection capability; Robust to multicollinearity	More hyperparameters to tune; Computationally more expensive; Still assumes linearity	Correlated features + feature selection; High-dimensional data; Genomics, finance
Random Forest	Ensemble of decision trees trained on random subsets of data and features.	Handles non-linear relationships; Resistant to overfitting; No feature scaling needed; Provides feature importance; Handles missing values	Less interpretable; Slower prediction; Can be memory-intensive; May overfit on noisy data	Complex non-linear relationships; Mixed feature types; Feature importance needed; General-purpose regression
XGBoost	Optimized gradient boosting implementation with regularization.	High accuracy; Fast training; Built-in regularization; Handles missing values; Feature importance; Parallel processing	Sensitive to hyperparameters; Prone to overfitting with wrong settings; Less interpretable; Requires careful tuning	Kaggle competitions; Structured/tabular data; High-performance needs; Industry standard
LightGBM	Gradient boosting using leaf-wise tree growth (faster than XGBoost).	Very fast training; Memory efficient; Handles large datasets; High accuracy; Supports categorical features	Can overfit small datasets; Requires careful parameter tuning; Less interpretable; Sensitive to overfitting	Large datasets (>10K rows); Fast training needed; Limited memory; Production environments
Gradient Boosting	Sequential ensemble building trees to correct previous errors.	High accuracy; Handles non-linearity; Feature importance; Flexible loss functions	Slow training; Prone to overfitting; Requires careful tuning; Sequential (not parallelizable)	Structured data; High accuracy needed; Complex patterns; Competition-level performance
Support Vector	Uses kernel trick to	Effective in high	Slow on large datasets;	Small to medium

Regressor (SVR)	map data to higher dimensions for regression.	dimensions; Works with non-linear data; Robust to outliers; Memory efficient (uses support vectors)	Sensitive to feature scaling; Difficult hyperparameter tuning; Not probabilistic	datasets; High-dimensional space; Non-linear relationships; Robust predictions needed
Decision Tree	Tree-based model splitting data based on feature thresholds.	Highly interpretable; No feature scaling needed; Handles non-linearity; Fast prediction; Handles mixed data types	Prone to overfitting; Unstable (high variance); Biased toward features with more levels; Poor extrapolation	Interpretability critical; Quick baseline; Data exploration; Rule extraction
CatBoost	Gradient boosting optimized for categorical features.	Excellent with categorical features; Reduces overfitting; Fast training; Minimal preprocessing; Good default parameters	Slower than LightGBM; Memory intensive; Less community support; Newer (less tested)	Many categorical features; Mixed data types; Minimal preprocessing desired; Robust to overfitting
K-Nearest Neighbors (KNN)	Predicts based on average of K nearest training samples.	No training phase; Simple and intuitive; Naturally handles multi-output; No assumptions about data	Slow prediction on large data; Memory intensive; Sensitive to feature scaling; Curse of dimensionality; Requires choosing K	Small datasets; Low dimensions; Smooth decision boundaries; Instance-based learning
AdaBoost	Boosting algorithm giving more weight to misclassified samples.	Less prone to overfitting than single trees; No hyperparameter tuning needed; Feature selection capability; Interpretable	Sensitive to outliers; Slower than Random Forest; Can underperform with noisy data; Sequential training	Clean datasets; Weak learners combination; Moderate-sized datasets; Less noisy data
Bayesian Ridge	Bayesian approach to ridge regression with automatic hyperparameter optimization.	Automatic regularization tuning; Provides uncertainty estimates; Robust to overfitting; No cross-validation needed	Assumes Gaussian prior; Computationally intensive; Still linear model; Complex interpretation	Uncertainty quantification needed; Small datasets; Automatic tuning desired; Probabilistic predictions
Huber Regressor	Linear regression optimized for robustness to outliers.	Robust to outliers; Better than OLS with contaminated data; Combines L1 and L2 loss; Interpretable	Still assumes linearity; Requires epsilon tuning; Slower than OLS; Less known/used	Data with outliers; Robust regression needed; Mixed noise types; Financial data
Neural Networks (MLPRegressor)	Multi-layer perceptron with hidden layers for complex patterns.	Learns complex non-linear patterns; Flexible architecture; Can approximate any function; Transfer learning possible	Requires large data; Computationally expensive; Black box; Sensitive to hyperparameters; Requires feature scaling	Very large datasets; Complex non-linear patterns; Image/text features; Deep learning needed
Polynomial	Linear regression	Captures non-linear	Prone to overfitting with	Known polynomial

Regression	with polynomial features.	relationships; Simple extension of linear regression; Interpretable with low degrees	high degrees; Extrapolation problems; Computationally expensive with many features	relationships; Small feature space; Curve fitting; Scientific modeling

- **Small data, interpretability**: Linear Regression, Ridge, Lasso
- **Large data, high accuracy**: XGBoost, LightGBM, CatBoost
- **Non-linear patterns**: Random Forest, Gradient Boosting, Neural Networks
- **Categorical features**: CatBoost, Random Forest
- **Outliers present**: Huber Regressor, Random Forest
- **Feature selection**: Lasso, ElasticNet

Some models that you can use:

1. XGBoost

- Gold standard for structured/tabular data in sports analytics
- Excellent feature importance for identifying market inefficiencies
- Handles missing values (e.g., incomplete player statistics)

2. LightGBM

- Much faster training if you have large datasets (>10K matches)
- Memory efficient for experimentation
- Similar accuracy to XGBoost with better speed

3. Logistic Regression

- Interpretable - you can see which features matter
- Fast and simple for initial analysis

4. Random Forest

- Less prone to overfitting than XGBoost (important for generalization)
- Handles non-linear patterns well

5. CatBoost

- Excellent if you have many categorical features (player names, surface type, tournament, weather)
- Minimal preprocessing required
- Less overfitting than XGBoost
- Good default parameters - less tuning needed

6. Gradient Boosting

- Simpler than XGBoost for initial hyperparameter tuning
- Feature importance analysis

Sample Workflow:

1. Baseline: logistic = LogisticRegression(), baseline_score = evaluate(logistic)
2. Tree ModelsL
rf = RandomForestClassifier(), xgb = XGBClassifier(), lgbm = LGBMClassifier(), catboost = CatBoostClassifier()
3. Compare: results = compare_models([logistic, rf, xgb, lgbm, catboost])
4. Ensemble top 3: ensemble = VotingClassifier([best_model_1, best_model_2, best_model_3])
5. Market Analysis: compare_with_bookmaker_odds(ensemble, odds_data)

(A VotingClassifier is a machine learning ensemble method that combines the predictions from multiple individual classifiers to produce a more accurate, robust, and generalized final prediction. It operates on the principle that the collective wisdom of diverse models often outperforms any single model alone)

check this out: (Regression Models Implementation Example)

```
# Split the dataset into training and testing subsets with random shuffling
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize and apply standard scaler to numerical features for normalization
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Dictionary of candidate regression models with default hyperparameters
model_candidates = {
    'Ridge Regression': Ridge(),
    'Lasso Regression': Lasso(),
    'ElasticNet': ElasticNet(),
    'Bayesian Ridge': BayesianRidge(),
    'Huber Regressor': HuberRegressor(),
    'Decision Tree': DecisionTreeRegressor(),
    'Random Forest': RandomForestRegressor(),
    'Gradient Boosting': GradientBoostingRegressor(),
    'AdaBoost': AdaBoostRegressor(),
    'K-Nearest Neighbors': KNeighborsRegressor(),
    'Support Vector Regressor': SVR(),
    'XGBoost': XGBRegressor(verbosity=0),
    'LightGBM': LGBMRegressor(verbose=-1),
    'CatBoost': CatBoostRegressor(verbose=0)
}

# Hyperparameter grids for GridSearch cross-validation
param_grids = {
    'Ridge Regression': {'alpha': [0.1, 1.0, 10.0]},
    'Lasso Regression': {'alpha': [0.1, 1.0, 10.0]},
    'ElasticNet': {'alpha': [0.1, 1.0, 10.0], 'l1_ratio': [0.1, 0.5, 0.9]},
    'Bayesian Ridge': {
        'alpha_1': [1e-6, 1e-5, 1e-4], 'alpha_2': [1e-6, 1e-5, 1e-4],
        'lambda_1': [1e-6, 1e-5, 1e-4], 'lambda_2': [1e-6, 1e-5, 1e-4]
    },
    'Huber Regressor': {'alpha': [0.0001, 0.001, 0.01]},
    'Decision Tree': {'max_depth': [None, 10, 20, 30]},
    'Random Forest': {'n_estimators': [100, 200, 500], 'max_depth': [None, 10, 20, 30]},
    'Gradient Boosting': {'n_estimators': [100, 200, 500], 'learning_rate': [0.01, 0.1, 0.2]},
    'AdaBoost': {'n_estimators': [50, 100, 200], 'learning_rate': [0.01, 0.1, 0.2]},
    'K-Nearest Neighbors': {'n_neighbors': [3, 5, 10], 'weights': ['uniform', 'distance']},
    'Support Vector Regressor': {'kernel': ['linear', 'rbf'], 'C': [0.1, 1.0, 10.0], 'epsilon': [0.01, 0.1
, 0.2]},
    'XGBoost': {'n_estimators': [100, 200, 500], 'learning_rate': [0.01, 0.1, 0.2]},
    'LightGBM': {'n_estimators': [100, 200, 500], 'learning_rate': [0.01, 0.1, 0.2]},
    'CatBoost': {'iterations': [100, 200, 500], 'learning_rate': [0.01, 0.1, 0.2]}
}

# Helper function to compute adjusted R²
def adjusted_r2(r2, n_samples, n_features):
```

```
    return 1 - (1 - r2) * (n_samples - 1) / (n_samples - n_features - 1)

# Initialize dictionary to store results and predictions
results = {
    'Model': [],
    'MSE_Train': [],
    'R2_Train': [],
    'Adj_R2_Train': [],
    'MAE_Train': [],
    'MSE_Test': [],
    'R2_Test': [],
    'Adj_R2_Test': [],
    'MAE_Test': [],
    'y_test_true': [],
    'y_test_pred': [],
    'y_train_true': [],
    'y_train_pred': []
}

# Train, tune, predict, and evaluate models
for name, model in model_candidates.items():
    print(f"Training and tuning {name}...")

    if name in param_grids:
        grid_search = GridSearchCV(model, param_grids[name], scoring='neg_mean_squared_err
or', cv=5, n_jobs=-1, verbose=0)
        grid_search.fit(X_train_scaled, y_train)
        best_model = grid_search.best_estimator_
    else:
        best_model = model
        best_model.fit(X_train_scaled, y_train)

    y_train_pred = best_model.predict(X_train_scaled)
    y_test_pred = best_model.predict(X_test_scaled)

    mse_train = mean_squared_error(y_train, y_train_pred)
    r2_train = r2_score(y_train, y_train_pred)
    adj_r2_train = adjusted_r2(r2_train, len(y_train), X_train_scaled.shape[1])
    mae_train = mean_absolute_error(y_train, y_train_pred)

    mse_test = mean_squared_error(y_test, y_test_pred)
    r2_test = r2_score(y_test, y_test_pred)
    adj_r2_test = adjusted_r2(r2_test, len(y_test), X_test_scaled.shape[1])
    mae_test = mean_absolute_error(y_test, y_test_pred)

    results['Model'].append(name)
    results['MSE_Train'].append(mse_train)
    results['R2_Train'].append(r2_train)
    results['Adj_R2_Train'].append(adj_r2_train)
    results['MAE_Train'].append(mae_train)
    results['MSE_Test'].append(mse_test)
```

```
        results['R2_Test'].append(r2_test)
        results['Adj_R2_Test'].append(adj_r2_test)
        results['MAE_Test'].append(mae_test)
        results['y_test_true'].append(y_test)
        results['y_test_pred'].append(y_test_pred)
        results['y_train_true'].append(y_train)
        results['y_train_pred'].append(y_train_pred)

# Convert to DataFrame and sort by test R2
results_df = pd.DataFrame(results).sort_values(by='R2_Test', ascending=False).reset_index(drop=Tru
e)

# --------- Plot MAE Train vs Test ----------
plt.figure(figsize=(12,6))
sns.scatterplot(x='MAE_Train', y='MAE_Test', data=results_df, hue='Model', palette=custom_palette,
s=100)
plt.plot([results_df['MAE_Train'].min(), results_df['MAE_Train'].max()],
                [results_df['MAE_Train'].min(), results_df['MAE_Train'].max()],
                'r--', lw=2)
plt.title('Mean Absolute Error (Train vs Test)', fontsize=16)
plt.xlabel('MAE Train')
plt.ylabel('MAE Test')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True)
plt.show()

# --------- Plot R2 Train vs Test ----------
plt.figure(figsize=(12,6))
sns.scatterplot(x='R2_Train', y='R2_Test', data=results_df, hue='Model', palette=custom_palette, s=1
00)
plt.plot([results_df['R2_Train'].min(), results_df['R2_Train'].max()],
                [results_df['R2_Train'].min(), results_df['R2_Train'].max()],
                'r--', lw=2)
plt.title('R² Score (Train vs Test)', fontsize=16)
plt.xlabel('R² Train')
plt.ylabel('R² Test')

plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True)
plt.show()

# --------- Scatter plot of Actual/Predicted Ratio (Test data) ----------
plt.figure(figsize=(14,8))
ratios = []
models_expanded = []
for i, row in results_df.iterrows():
        ratio = row['y_test_true'] / (row['y_test_pred'] + 1e-8)    # Avoid div by zero
        ratios.extend(ratio)
        models_expanded.extend([row['Model']]*len(ratio))

df_ratio = pd.DataFrame({'Ratio': ratios, 'Model': models_expanded})
```

```
sns.histplot(data=df_ratio, x='Ratio', hue='Model', element='step', stat='count', palette=custom_pale
tte, bins=30)
plt.title('Distribution of Actual/Predicted Ratio by Model (Test Data)', fontsize=16)
plt.xlabel('Actual / Predicted')
plt.ylabel('Count')
plt.grid(True)
plt.show()

# --------- Residuals distribution plot (Test data) ----------
plt.figure(figsize=(14,8))
for i, row in results_df.iterrows():
        residuals = row['y_test_true'] - row['y_test_pred']
        sns.kdeplot(residuals, label=row['Model'], fill=True, alpha=0.3)
plt.title('Residuals Distribution (Test Data)', fontsize=16)
plt.xlabel('Residuals (Actual - Predicted)')

plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True)
plt.show()

# --------- Actual vs Predicted plot for all models ----------
fig, axs = plt.subplots(len(results_df), 1, figsize=(10, len(results_df)*4), sharex=True)
if len(results_df) == 1:
        axs = [axs]

for i, row in results_df.iterrows():
        sns.scatterplot(x=row['y_test_true'], y=row['y_test_pred'], ax=axs[i], color=custom_palette[i %
len(custom_palette)], alpha=0.6)
        axs[i].plot([row['y_test_true'].min(), row['y_test_true'].max()], [row['y_test_true'].min(), row['y_
test_true'].max()], 'r--')
        axs[i].set_title(f'Actual vs Predicted Premium - {row["Model"]}')
        axs[i].set_xlabel('Actual Premium')
        axs[i].set_ylabel('Predicted Premium')
        axs[i].grid(True)

plt.tight_layout()
plt.show()

# Custom palette
custom_palette = ['#B7D5D4', '#77878B', '#488286', '#305252']
custom_cmap = LinearSegmentedColormap.from_list("custom", custom_palette)

# --------- Styled Performance Summary Table ---------
styled_results = results_df[['Model', 'MSE_Train', 'MAE_Train', 'R2_Train', 'Adj_R2_Train',
                                        'MSE_Test', 'MAE_Test', 'R2_Test', 'Adj_R2_Test']].style\
        .background_gradient(cmap=custom_cmap, axis=1)\
        .format({
                'MSE_Train': "{:.2f}",
                'MAE_Train': "{:.2f}",
                'R2_Train': "{:.3f}",
                'Adj_R2_Train': "{:.3f}",
```

```
            'MSE_Test': "{:.2f}",
            'MAE_Test': "{:.2f}",
            'R2_Test': "{:.3f}",
            'Adj_R2_Test': "{:.3f}",
        })\
        .set_caption("Comprehensive Regression Models Performance Summary - Target: Expenses")

display(styled_results)


# Additional line plot for train vs test R2 scores
plt.figure(figsize=(14, 7))
df_long = pd.melt(results_df[['Model', 'R2_Train', 'R2_Test']], id_vars='Model',
                  value_vars=['R2_Train', 'R2_Test'],
                  var_name='Dataset', value_name='R2_Score')

palette = ['#488286', '#B7D5D4']

sns.lineplot(data=df_long, x='Model', y='R2_Score', hue='Dataset', marker='o', palette=palette)
plt.xticks(rotation=45, fontsize=12)
plt.title('R² Score Comparison: Train vs Test for All Models')
plt.xlabel('Model')
plt.ylabel('R² Score')
plt.legend(title='Dataset')
plt.grid(True)
plt.tight_layout()
plt.show()

# Additional line plot for MAE comparison
plt.figure(figsize=(14, 6))
plt.plot(results_df['Model'], results_df['MAE_Train'], marker='o', label='Train MAE', color='#B7D5D4'
)
plt.plot(results_df['Model'], results_df['MAE_Test'], marker='o', label='Test MAE', color='#305252')
plt.xticks(rotation=45, ha='right')
plt.title('MAE Comparison Across Models')
plt.xlabel('Models')
plt.ylabel('Mean Absolute Error')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```